parser.y línea a línea

Los números de línea son los del `parser.y` actual (301 líneas). Abridlo al lado y seguid el orden.

Líneas 1–17: cabecera y código C inicial

Línea	Código	Qué hace
1–6	`//Archivo: parser.y` …	Comentario de cabecera: qué es el fichero, en qué paso estamos y autores. Empiezan con un espacio para que bison no los confunda con una directiva
8–9	`//PARTE 1: ...`	Comentarios que marcan la zona de declaraciones, igual que en el ejemplo del profesor
10	`%{`	Abre un bloque de C. Todo lo que hay hasta `%}` se copia tal cual al principio de `parser.tab.c`
11	`#include <stdio.h>`	Para usar `printf`, `fopen` y el tipo `FILE`
12	`#include <stdlib.h>`	Para `EXIT_SUCCESS` y `EXIT_FAILURE`
13	`int yylex(void);`	Prototipo del scanner. La función la escribe flex en `lex.yy.c`; aquí solo se avisa al compilador de C de que existe, porque el parser la llama
14	`void yyerror(const char *);`	Prototipo de la función de error. Bison la llama cuando un token no encaja; la escribimos nosotros en la línea 299
15	`extern FILE* yyin;`	`yyin` es el fichero del que lee flex. Está definido en `lex.yy.c`; `extern` dice "existe en otro fichero". El `main` la usa para abrir el `.alg`
16	`extern int yylineno;`	La línea por la que va el scanner (la activa `%option yylineno` en `scanner.l`). La usamos para decir dónde está el error
17	`%}`	Cierra el bloque de C

Líneas 19–28: mensajes de error y `%union`

Línea	Código	Qué hace
20	`%define parse.error verbose`	Hace que el mensaje de error diga qué token llegó y cuál esperaba (`unexpected identificadorTK, expecting puntoYComaTK`). Sin ella solo diría `syntax error`
23	`%union{`	Define el tipo de `yylval`, la variable por la que el scanner le pasa al parser el valor de un token. Es un `union` de C: guarda **uno** de estos campos cada vez
24	`char * cadena;`	Para los identificadores (el nombre, por ejemplo `aux`) y los literales cadena
25	`int entero;`	Para los literales enteros (`42`)
26	`float real;`	Para los literales reales (`3.5`)
27	`char caracter;`	Para los literales carácter (`"a"`)
28	`}`	Cierra el `%union`

Ejemplo: cuando el scanner lee `aux` hace `yylval.cadena = strdup("aux"); return identificadorTK;`. El parser recibe el token y el nombre.

Líneas 30–50: los tokens

`%token` declara los terminales de la gramática. Bison les da un número y los escribe en `parser.tab.h`; `scanner.l` incluye ese fichero, y así `return algoritmoTK;` en el scanner y `algoritmoTK` en el parser son el mismo número.

Línea	Código	Qué hace
32–36	`%token accionTK algoritmoTK ...`	Una por cada palabra reservada del enunciado. En una línea `%token` caben varios nombres, por eso están agrupadas
38	`%token precondicionTK postcondicionTK`	Los comentarios `{Prec...}` y `{Post...}`. Son tokens porque la sintaxis los exige; el resto de comentarios los ignora el scanner

Línea	Código	Qué hace
40	`%token <cadena> identificadorTK`	Token **multivaluado**: además del tipo, lleva un valor (el nombre). `<cadena>` dice en qué campo del `%union` va
41	`%token <entero> literalEnteroTK`	Su valor es el número, en el campo `entero`
42	`%token <real> literalRealTK`	Su valor en el campo `real`
43	`%token <caracter> literalCaracterTK`	Su valor en el campo `caracter`
44	`%token <cadena> literalCadenaTK`	Su valor (el texto) en el campo `cadena`
46	`asignacionTK subrangoTK entoncesTK sinoSiTK`	`:=`, `..`, `->` y `[]`
47	`menorTK mayorTK igualTK ...`	Los relacionales: `<`, `>`, `=`, `<=`, `>=`, `<>`
48	`masTK menosTK porTK divRealTK`	`+`, `-`, `*`, `/` (`div` y `mod` son palabras reservadas, línea 33–35)
49	`parIzqTK parDerTK corchIzqTK corchDerTK`	`(`, `)`, `[`, `]`
50	`comaTK puntoYComaTK puntoTK dosPuntosTK`	`,`, `;`, `.`, `:`

Líneas 52–63: prioridades y símbolo inicial

Estas líneas resuelven la ambigüedad de las expresiones. **Cada línea tiene más prioridad que la de arriba.** La palabra del principio dice la asociatividad.

Línea	Código	Qué hace
54	`%left oTK`	`o` es lo que menos prioridad tiene: `a y b o c` es `(a y b) o c`. `%left`: `a o b o c` es `(a o b) o c`
55	`%left yTK`	`y` va antes que `o`, como el producto antes que la suma
56	`%right noTK`	`no` es unario. Está por debajo de los relacionales, así que `no a < b` es `no (a < b)`

Línea	Código	Qué hace
57	`%nonassoc igualTK ...`	Los seis relacionales con la misma prioridad. `%nonassoc` = no se pueden encadenar: `a < b < c` da error
58	`%left masTK menosTK`	Suma y resta. `a - b - c` es `(a - b) - c`
59	`%left porTK divRealTK divTK modTK`	`*`, `/`, `div`, `mod`: más prioridad que la suma, así que `a + b * c` es `a + (b * c)`
60	`%right UNARIO`	`UNARIO` no es un token: es un nombre para una prioridad. Las reglas del `-` y `+` unarios la usan con `%prec UNARIO` (líneas 187–188), así que `-a * b` es `(-a) * b`
61	`%left puntoTK corchIzqTK refTK`	Lo más prioritario: `.campo`, `[índice]` y `ref` se pegan al operando antes que cualquier operador
63	`%start descAlgoritmo`	El símbolo inicial (axioma). Si no se pone, bison usa la primera regla, que aquí es la misma
65	`%%`	Fin de las declaraciones; empiezan las reglas

Líneas 65–96: la estructura del programa

A partir de aquí cada bloque es una regla: `nombre`, luego `:` con la primera alternativa, `|` para cada alternativa más, y `;` para cerrar. `/* vacio */` es la alternativa ε.

Línea	Regla	Qué reconoce
69–71	`descAlgoritmo`	El programa entero: `algoritmo nombre ;`, la cabecera, el bloque y `falgoritmo .`
72–74	`cabeceraAlg`	Lo que va antes del código: tipos y constantes globales, acciones y funciones, variables `ent` / `sal` y la precondición
75–77	`bloqueAlg`	El cuerpo del algoritmo seguido de la postcondición
78–82	`declGlobales`	Cero o más declaraciones de `tipo` o `const`. Es recursiva: una declaración seguida de más `declGlobales`, hasta llegar al vacío

Línea	Regla	Qué reconoce
83–87	`declAF`	Cero o más acciones o funciones, con el mismo patrón
88–90	`bloque`	Declaraciones locales y después instrucciones. Se usa en el algoritmo y también dentro de acciones y funciones
91–96	`declaraciones`	Cero o más bloques `tipo`, `const` o `var`, en cualquier orden

Con `programa1.alg`: `algoritmo intercambio;` es la línea 70 hasta `puntoYComaTK`. `ent a, b: entero; sal a, b: entero; {Prec...}` es `cabeceraAlg` (sin globales ni acciones). `var aux: entero; fvar aux := a; ...` es el `bloque`, y `{Post...} falgoritmo.` cierra.

Líneas 98–172: declaraciones

Línea	Regla	Qué reconoce	Ejemplo
100–101	comentario	Explica por qué el `;` tras `ftipo` / `fconst` / `fvar` es opcional: el enunciado lo pide, pero sus ejemplos no lo ponen	
102–104	`declaracionTipo`	Un bloque de tipos	`tipo vector = tabla[1..10] de entero; ftipo`
105–107	`declaracionConst`	Un bloque de constantes	`const MAX = 10; fconst`
108–110	`declaracionVar`	Un bloque de variables	`var aux: entero; fvar`
111–114	`finDecl`	Un `;` o nada	
117–120	`listaDTipo`	Cero o más `nombre = tipo ;`	`vector = tabla[1..10] de entero;`

Línea	Regla	Qué reconoce	Ejemplo
121–128	`dTipo`	Cualquier tipo, una alternativa por línea: tupla (122), tabla (123), nombre de un tipo ya declarado (124), subrango (125), puntero con `ref` (126) o tipo básico (127)	`tupla cx: real;` `ftupla`, `1..10`, `ref entero`
129–135	`tipoBase`	Los cinco tipos predefinidos	`entero`, `booleano`, `caracter`, `real`, `cadena`
136–139	`listaCampos`	Los campos de una tupla, cero o más `campo : tipo ;`	`cx: real; cy: real;`
142–145	`listaDCte`	Cero o más `NOMBRE = literal ;`	`MAX = 10;`
146–153	`literal`	Cualquier valor constante: entero, real, carácter, cadena, `verdadero` o `falso`	`10`, `3.5`, `"a"`, `'hola'`
154–157	`listaDVar`	Cero o más `nombres : tipo ;`	`a, b: entero;`
158–161	`listaId`	Uno o más nombres separados por comas. Recursiva por la derecha: `a , (b , (c))`	`a, b, c`
162–166	`declEntSal`	`ent`, `ent` y `sal`, o solo `sal`. Al menos una de las dos	
167–169	`declEnt`	`ent` seguido de variables	`ent a, b: entero;`
170–172	`declSal`	`sal` seguido de variables	`sal max, min: entero;`

En la línea 123 el tamaño de una tabla y en la 125 un subrango se escriben con `expresion`, porque los límites pueden ser constantes o caracteres (`"a".."z"`).

Líneas 174–206: expresiones

Las líneas 174–179 son un comentario: explican que `exp_a` y `exp_b` del enunciado se han unido en una sola `expresion`, porque una variable puede ser de los dos tipos y el parser no sabría a cuál reducir.

Línea	Alternativa	Qué reconoce
181–186	`expresion masTK expresion`, `menosTK`, `porTK`, `divRealTK`, `divTK`, `modTK`	Las operaciones aritméticas binarias: `a + b`, `a - b`, `a * b`, `a / b`, `a div b`, `a mod b`
187	`menosTK expresion %prec UNARIO`	El menos unario: `-a`. `%prec UNARIO` le da la prioridad de la línea 60 en vez de la del menos binario
188	`masTK expresion %prec UNARIO`	El más unario: `+a`
189–190	`expresion yTK expresion`, `oTK`	Conjunción y disyunción: `p y q`, `p o q`
191	`noTK expresion`	Negación: `no p`
192	`expresion oprel expresion %prec igualTK`	Una comparación: `a >= b`. Como el operador está dentro de `oprel` (un no terminal), bison no ve su prioridad; `%prec igualTK` le da la de la línea 57
193	`parIzqTK expresion parDerTK`	Paréntesis: `(a + b)`
194	`operando`	Una variable o parte de ella
195	`literal`	Un valor constante (regla de la línea 146)
196	`funcionLl`	Una llamada a función: `doble(i)`
198–200	`oprel`	Los seis relacionales, en una sola línea separados por barras
201–206	`operando`	Una variable (202), un campo de tupla `p.cx` (203), un elemento de tabla `v[i]` (204) o lo apuntado por un puntero `q ref` (205). Es recursiva por la izquierda, así que se pueden encadenar: `v[i].cx`

Líneas 208–246: instrucciones

Línea	Regla	Qué reconoce	Ejemplo
210–211	comentario	El `;` separa instrucciones y se admite uno al final	
212–215	`instrucciones`	Una lista de instrucciones, con o sin `;` final	`aux := a; a := b; b := aux`
216–219	`listaInstrucciones`	Una o más instrucciones separadas por `;`. Recursiva por la izquierda (la lista va delante), que en bison gasta menos pila	
220–226	`instruccion`	Uno de los cinco tipos de instrucción	
221	`continuarTK`	La instrucción que no hace nada	`continuar`
227–229	`asignacion`	Variable, `:=` y expresión	`aux := a`
230–232	`alternativa`	`si`, condición, `->`, instrucciones, más opciones y `fsi`	`si a >= b -> max := a ... fsi`
233–236	`listaOpciones`	Cero o más `[] condición -> instrucciones`	`[] b < c -> med := c`
237–240	`iteracion`	Un bucle `para` o un bucle `mientras`	
241–243	`itCotaExp`	El bucle `mientras`	`mientras i < n hacer ... fmientras`
244–246	`itCotaFija`	El bucle `para`	`para i := 1 hasta n hacer ... fpara`

Una instrucción que empieza por un nombre puede ser una asignación (`a := ...`) o una llamada a acción (`incr(...)`). El parser lo decide mirando el token siguiente: `:=` o `(`.

Líneas 248–280: acciones y funciones

Línea	Regla	Qué reconoce	Ejemplo
250–252	`accionD`	La definición de una acción: cabecera, bloque y `faccion`	`accion incr(...); v := v + d faccion`
253–255	`funcionD`	La definición de una función: cabecera, bloque, `dev` con el valor que devuelve y `ffuncion`	`funcion doble(...) dev entero; ... dev r ffuncion`
256–258	`aCabecera`	Nombre y parámetros de una acción, entre paréntesis y con `;` final	`incr(e/s v: entero; ent d: entero;);`
259–261	`fCabecera`	Nombre, parámetros, `dev` y el tipo que devuelve. Los parámetros usan `listaDVar` (línea 154) porque en funciones son siempre de entrada y no se pone `ent`	`doble(n: entero;) dev entero;`
262–265	`dParForm`	Cero o más grupos de parámetros, cada uno terminado en `;`	
266–270	`dPForm`	Un grupo de parámetros: `ent`, `sal` o `e/s`, los nombres, `:` y el tipo	`e/s v: entero`
271–273	`accionLl`	Una llamada a acción (como instrucción)	`incr(s, 1)`
274–276	`funcionLl`	Una llamada a función (dentro de una expresión). Se escribe igual que `accionLl`, pero aparece en otro sitio, así que no hay conflicto	`doble(i)`
277–280	`lLl`	Los argumentos: una o más expresiones separadas por comas	`s, 1`
282	`%%`	Fin de las reglas; empieza el código C	

Líneas 282–301: `main` y `yyerror`

Esto es C normal; bison lo copia al final de `parser.tab.c`.

Línea	Código	Qué hace
284	`int main(int argc, char **argv){`	El programa empieza aquí. `argc` es cuántos argumentos hay y `argv` cuáles (`argv[1]` es el `.alg`)
285	`if (argc > 1) {`	¿Nos han pasado un fichero?
286	`if ((yyin = fopen(argv[1], "r")) == NULL) {`	Lo abre para leer y lo deja en `yyin`, de donde lee el scanner. Si `fopen` devuelve `NULL`, no se ha podido abrir
287–288	`printf(...); return EXIT_FAILURE;`	Avisa y termina con código de error
290–291	`} else yyin = stdin;`	Sin fichero, lee del teclado (se termina con Ctrl+D)
292	`if (yyparse() == 0) {`	Arranca el parser. `yyparse()` pide tokens a `yylex()` hasta el final y devuelve 0 si el programa es correcto
293–294	`printf("COMPILADOR: ..."); return EXIT_SUCCESS;`	Programa correcto: lo dice y termina bien
296	`return EXIT_FAILURE;`	`yyparse()` devolvió 1: hubo un error (ya lo ha contado `yyerror`)
299	`void yyerror(const char * s){`	La llama bison al ver un token que no encaja. `s` es el mensaje (`syntax error, unexpected ...`)
300	`printf("\tPARSER: ERROR en la linea %d: %s\n", yylineno, s);`	Escribe la línea del error (`yylineno`, de flex) y el mensaje de bison

En el paso 2 aquí se añadirá la creación de la tabla de símbolos antes de `yyparse()` y su impresión después, como hace el ejemplo del profesor con la tabla de constantes.